

# SymPy Bug Report

## 1 Bug 30: solveset over the reals treats a principal fractional-power identity as true for negative inputs

### 1.1 Minimal Reproducer

```
from sympy import Eq, N, Rational, S, solveset, symbols

x = symbols("x")
eq = Eq((x**2)**Rational(1, 3), x**Rational(2, 3))
sol = solveset(eq, x, S.Reals)

print("solveset(Eq((x**2)**Rational(1, 3), x**Rational(2, 3)), x, S.Reals) =", sol)
print("lhs at x = -1:", eq.lhs.subs(x, -1))
print("rhs at x = -1:", eq.rhs.subs(x, -1))
print("numeric residual at x = -1:", N(eq.lhs.subs(x, -1) - eq.rhs.subs(x, -1), 30))
```

### 1.2 Observed Behavior

```
SymPy version: 1.14.0
solveset(Eq((x**2)**Rational(1, 3), x**Rational(2, 3)), x, S.Reals) = Reals
lhs at x = -1: 1
rhs at x = -1: (-1)**(2/3)
numeric residual at x = -1: 1.5 - 0.866025403784438646763723170753*I
```

SymPy reports that every real number solves

$$(x^2)^{1/3} = x^{2/3}.$$

This is not a harmless set representation issue: the returned set contains  $x = -1$ , but direct substitution gives a nonzero residual.

### 1.3 Expected Behavior

The real solution set should exclude negative real numbers. In the representative case, the expected result is

$$[0, \infty).$$

Equivalently,  $x = -1$  and every other negative real value should not be reported as a solution.

## 1.4 Mathematical Explanation

SymPy's rational powers use principal complex powers. For  $x < 0$ , write  $x = -a$  with  $a > 0$ . Then

$$(x^2)^{1/3} = a^{2/3},$$

which is positive real, while

$$x^{2/3} = \exp\left(\frac{2}{3} \operatorname{Log}(-a)\right) = a^{2/3} \exp\left(\frac{2\pi i}{3}\right)$$

using the principal logarithm. These two values are not equal. In particular,

$$((-1)^2)^{1/3} = 1 \quad \text{but} \quad (-1)^{2/3} = \exp(2\pi i/3),$$

so the equation fails at  $x = -1$ . The identity  $(x^2)^{1/3} = x^{2/3}$  is therefore valid on the nonnegative real branch, but not on all of  $S.\operatorname{Reals}$ .

## 1.5 Source-Level Diagnosis

The source-level diagnosis identifies the relevant radical-elimination path through `sympy/solvers/solveset.py` and `sympy/solvers/solvers.py`. The public call enters `solveset`, normalizes the equality to

$$-x^{2/3} + (x^2)^{1/3},$$

and then calls `_solveset`. The radical path calls `unrad` and then `_solve_radical`.

The traced intermediate result is

```
unrad(-x**(2/3) + (x**2)**(1/3), x) -> (0, [])
solveset(0, x, S.Reals) -> Reals
```

The diagnosis locates the specific faulty transformation in `sympy/solvers/solvers.py`, around line 3535, inside `unrad`'s two-radical-term case:

```
eq = rterms[0]**lcm - (-rterms[1])**lcm
```

For this reproducer, `lcm` is 3. Cubing both sides rewrites the original equation as  $x^2 - x^2 = 0$ , so `unrad` returns the zero equation with no side conditions. That radical-elimination step assumes the principal-power identity is reversible over the real solving domain. It is not reversible for negative real  $x$ , because  $x^{2/3}$  is a principal complex power.

The diagnosis also notes that `_solve_radical` in `sympy/solvers/solveset.py` validates finite solution sets with `checksol`, but a whole-domain result such as  $S.\operatorname{Reals}$  is not checked pointwise and is accepted. Thus the branch condition  $x \geq 0$ , which is needed for the original principal-power equality, is lost. A plausible fix direction supported by the diagnosis is to make `unrad` branch-condition aware for rational powers, or, when those conditions cannot be produced, have `_solve_radical` return a conditional result rather than accepting a whole-domain superset.

## 1.6 Additional Failing Instances

The independent verification generalizes the representative equation to translated principal-power identities

$$((x+a)^2)^{1/3} = (x+a)^{2/3}$$

for  $a = 0, 1, 2, 3, 4, 5$ . For each value of  $a$ , SymPy returns a set containing the negative-branch witness  $x = -a - 1$ . At that witness,  $x + a = -1$ , so the same principal-branch contradiction appears: the left side is 1, while the right side is the principal value  $(-1)^{2/3}$ .

| Input / Expression                                        | SymPy behavior                    | Expected behavior                                  |
|-----------------------------------------------------------|-----------------------------------|----------------------------------------------------|
| $a = 0: (x^2)^{1/3} = x^{2/3}$ , witness $x = -1$         | returns $S.Reals$ ; contains $-1$ | exclude $-1$ ; representative result $[0, \infty)$ |
| $a = 1: ((x+1)^2)^{1/3} = (x+1)^{2/3}$ , witness $x = -2$ | contains $-2$                     | exclude $-2$                                       |
| $a = 2: ((x+2)^2)^{1/3} = (x+2)^{2/3}$ , witness $x = -3$ | contains $-3$                     | exclude $-3$                                       |
| $a = 3: ((x+3)^2)^{1/3} = (x+3)^{2/3}$ , witness $x = -4$ | contains $-4$                     | exclude $-4$                                       |
| $a = 4: ((x+4)^2)^{1/3} = (x+4)^{2/3}$ , witness $x = -5$ | contains $-5$                     | exclude $-5$                                       |
| $a = 5: ((x+5)^2)^{1/3} = (x+5)^{2/3}$ , witness $x = -6$ | contains $-6$                     | exclude $-6$                                       |

The expected behavior is the same across the family because all cases reduce to the same branch condition on  $y = x + a$ : the equality  $(y^2)^{1/3} = y^{2/3}$  holds for nonnegative real  $y$  and fails for negative real  $y$ .

## 1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check and classified this as a new instance of a known failure mode. The closest related public material identified in the artifact is SymPy's documentation for principal roots, SymPy's domain-sensitive `solveset` documentation, and general CAS-family literature on branch hazards for fractional powers. These sources describe the broad mathematical risk, but the available evidence did not identify a public report of this exact real-domain `solveset` result, this equation, or the negative-real counterexample for  $x^{2/3}$ . The case therefore remains individually actionable as a specific reproducer and regression test.

## 1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import Eq, N, Rational, S, solveset, symbols

@pytest.mark.parametrize("a", [0, 1, 2, 3, 4, 5])
def test_principal_fractional_power_equation_excludes_negative_branch(a):
    x = symbols("x")
    lhs = ((x + a)**2)**Rational(1, 3)
    rhs = (x + a)**Rational(2, 3)
    bad = -a - 1
    sol = solveset(Eq(lhs, rhs), x, domain=S.Reals)
    assert N((lhs - rhs).subs(x, bad), 30) != 0
    assert sol.contains(bad) is not S.true
```